# Configuring Yabai as a Tiling Window Manager on macOS

Ronald 'Ryy' G. Thomas

2025-06-20

## Table of contents

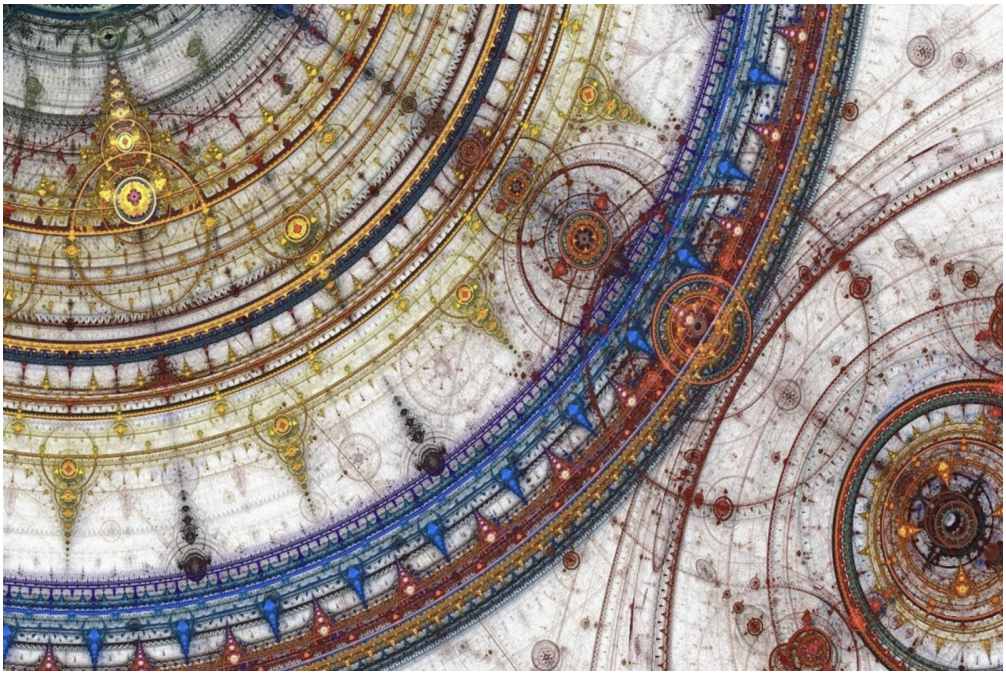

Figure 1: A clean desktop workspace with tiled application windows arranged in a grid layout.

*A tiled workspace where every pixel is accounted for and every window is exactly where it should be.*

# 1 Introduction

I did not really know how much faster a tiling window manager could make my daily workflow until I configured yabai with keyboard shortcuts and stopped reaching for the mouse. Before yabai, I spent a surprising amount of time dragging windows around, resizing them manually, and hunting for applications buried under overlapping panes.

The trigger was a morning where I counted how many times I reached for the mouse just to rearrange windows. It was north of fifty times in a single hour. Every mouse grab broke my concentration and interrupted whatever I was thinking about. I had heard of tiling window managers on Linux – i3wm, bspwm, dwm – but assumed macOS had nothing comparable.

Yabai proved me wrong. We document the complete configuration used: the `.yabairc` file that controls tiling behaviour, the `skhd` configuration that maps keyboard shortcuts to window actions, and the installation steps to get both tools running on macOS. The goal is a fully keyboard-driven desktop where windows arrange themselves automatically.

More formally, we document an optional addition to the Applications layer (Layer 10) of the Workflow Construct described in post 52. A tiling window manager is not load-bearing for the construct: nothing in the construct's other layers requires a tiled desktop, and many practitioners get equivalent productivity from the macOS default window manager plus keyboard-driven app switching. This post is presented as a layer-extension for users who want the additional ergonomics; it can be skipped without leaving gaps in the construct.

## 1.1 Motivations

- I was reaching for the mouse dozens of times per hour just to resize and reposition windows, and each interruption broke my concentration.
- My R development workflow involved switching between a terminal, a browser, a PDF viewer, and Vim, and overlapping windows meant I could never see more than one application at a time.
- I had used i3wm briefly on a Linux machine and was impressed by how much faster keyboard-driven window management felt compared to the drag-and-drop model.
- The macOS built-in window management (Split View, Mission Control) felt slow and limited, offering only two-pane splits with no fine-grained control.
- I wanted a configuration file I could version control and deploy on any macOS machine, rather than relying on GUI preferences that were difficult to reproduce.

## 1.2 Objectives

1. Install yabai and skhd on macOS using Homebrew and configure both services to start automatically.
2. Write a complete `.yabairc` configuration file with binary space partitioning layout, window gaps, border colours, and mouse integration.
3. Write a complete `.skhdrc` configuration file with keyboard shortcuts for navigation, window movement, resizing, workspace management, and layout rotation.
4. Document the full keybinding reference so that the configuration is self-explanatory.

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

*A focused workspace starts with removing visual clutter.*

## 2 Prerequisites and Setup

Before proceeding, the following are required:

- **macOS 11 (Big Sur)** or later
- **Homebrew** package manager installed
- **System Integrity Protection (SIP)** partially disabled (required for yabai's scripting additions)
- Basic comfort with the terminal and editing configuration files

Yabai requires elevated permissions to manage windows across all applications. The scripting addition (`sudo yabai --load-sa`) needs SIP to be partially disabled. Consult the yabai wiki for detailed instructions on this step.

## 3 What is a Tiling Window Manager?

A tiling window manager is a system that organises screen space into mutually non-overlapping frames. Instead of the standard macOS approach where windows float on top of one another and the user drags them into position, a tiling manager automatically assigns each window to a portion of the screen. When a new window opens, the existing layout adjusts to accommodate it.

Think of it like a bookshelf versus a pile of books on a desk. With a stacking window manager (the macOS default), windows pile up and one digs through the stack to find what is needed. With a tiling manager, every window has its own designated slot, and everything is visible at once.

Yabai is specifically a binary space partitioning (BSP) tiling manager. When a new window appears, yabai splits the current space in half (either horizontally or vertically) and assigns the new window to one half. The result is a tree of non-overlapping rectangles that fills the entire screen.

## 4 Getting Started

Installation uses Homebrew. The following commands install yabai and start it as a background service.

### 4.1 Installation

```
brew install koekeishiya/formulae/yabai
```

Start the yabai service:

```
yabai --start-service
```

To restart after changing the configuration:

```
yabai --restart-service
```

Keyboard shortcuts require a separate tool. The standard companion is `skhd` (Simple Hotkey Daemon):

```
brew install koekeishiya/formulae/skhd
skhd --start-service
```

With both services running, yabai manages window layout and skhd translates keyboard shortcuts into yabai commands.

Figure 2: Hands on a keyboard with a tiled desktop visible on the monitor in the background.

*The keyboard becomes the primary interface for all window operations.*

## 5 The Complete .yabairc Configuration

Create the file `~/.yabairc` (or wherever the yabai configuration resides). This file is a bash script that yabai sources on startup. It defines the tiling layout, gap sizes, border colours, and mouse behaviour.

```
#!/usr/bin/env bash

# Load scripting addition for window management
sudo yabai --load-sa
yabai -m signal --add \
  event=dock_did_restart \
  action="sudo yabai --load-sa"

set -x

# ====== Variables ============================

declare -A gaps
declare -A color

gaps["top"]="12"
gaps["bottom"]="12"
gaps["left"]="12"
gaps["right"]="12"
gaps["inner"]="12"

color["focused"]="0xE0808080"
color["normal"]="0x00010101"
color["preselect"]="0xE02d74da"

# ====== Tiling Settings ======================

yabai -m config layout bsp

yabai -m config top_padding \
  "${gaps["top"]}"
yabai -m config bottom_padding \
  "${gaps["bottom"]}"
yabai -m config left_padding \
  "${gaps["left"]}"
yabai -m config right_padding \
  "${gaps["right"]}"
yabai -m config window_gap \
```

```
  "${gaps["inner"]}"

yabai -m config mouse_follows_focus on
yabai -m config focus_follows_mouse on

yabai -m config window_topmost    off
yabai -m config window_opacity    off
yabai -m config window_shadow     float

yabai -m config window_border     on
yabai -m config window_border_width 2
yabai -m config active_window_border_color \
  "${color["focused"]}"
yabai -m config normal_window_border_color \
  "${color["normal"]}"
yabai -m config insert_feedback_color \
  "${color["preselect"]}"

yabai -m config active_window_opacity  1.0
yabai -m config normal_window_opacity  0.90
yabai -m config split_ratio            0.50

yabai -m config auto_balance on

yabai -m config mouse_modifier  fn
yabai -m config mouse_action1   move
yabai -m config mouse_action2   resize

set +x
printf "yabai: configuration loaded...\n"
```

The key settings to understand:

- **`layout bsp`** activates binary space partitioning, which is the core tiling behaviour.
- **`gaps`** control the pixel padding between windows and screen edges. Twelve pixels provides enough visual separation to distinguish windows without wasting space.
- **`mouse_follows_focus`** and **`focus_follows_mouse`** together create a seamless experience where the cursor and window focus stay synchronised.
- **`auto_balance on`** equalises window sizes after every layout change, preventing one window from dominating the screen.
- **`mouse_modifier fn`** allows holding the `fn` key to move (`mouse_action1`) or resize (`mouse_action2`) windows with the mouse when needed.

# 6 The Complete skhd Configuration

Create the file `~/.skhdrc`. This file maps keyboard shortcuts to yabai commands. The configuration below covers navigation, window movement, workspace management, resizing, layout rotation, and service control.

## 6.1 Window Navigation

These shortcuts move focus between windows using Vim-style directional keys (`h`, `j`, `k`, `l`) with the `Alt` modifier.

```
# Navigation: move focus between windows
alt - h : yabai -m window --focus west
alt - j : yabai -m window --focus south
alt - k : yabai -m window --focus north
alt - l : yabai -m window --focus east
```

| Shortcut | Action |
|---|---|
| `Alt-H` | Focus window to the left |
| `Alt-J` | Focus window below |
| `Alt-K` | Focus window above |
| `Alt-L` | Focus window to the right |

## 6.2 Window Movement (Warping)

These shortcuts physically move the focused window to a new position in the layout using `Shift-Alt` and directional keys.

```
# Moving windows: warp focused window
shift + alt - h : \
  yabai -m window --warp west
shift + alt - j : \
  yabai -m window --warp south
shift + alt - k : \
  yabai -m window --warp north
shift + alt - l : \
  yabai -m window --warp east
```

| Shortcut | Action |
|---|---|
| `Shift-Alt-H` | Warp window left |
| `Shift-Alt-J` | Warp window down |
| `Shift-Alt-K` | Warp window up |
| `Shift-Alt-L` | Warp window right |

### 6.3 Workspace Management

Move the focused window to a specific workspace (space) and follow the focus to that workspace.

```
# Move window to workspace and follow
shift + alt - m : \
  yabai -m window --space last; \
  yabai -m space --focus last
shift + alt - p : \
  yabai -m window --space prev; \
  yabai -m space --focus prev
shift + alt - n : \
  yabai -m window --space next; \
  yabai -m space --focus next
shift + alt - 1 : \
  yabai -m window --space 1; \
  yabai -m space --focus 1
shift + alt - 2 : \
  yabai -m window --space 2; \
  yabai -m space --focus 2
shift + alt - 3 : \
  yabai -m window --space 3; \
  yabai -m space --focus 3
shift + alt - 4 : \
  yabai -m window --space 4; \
  yabai -m space --focus 4
```

| Shortcut | Action |
|---|---|
| `Shift-Alt-M` | Move to last workspace |
| `Shift-Alt-P` | Move to previous workspace |
| `Shift-Alt-N` | Move to next workspace |
| `Shift-Alt-1` | Move to workspace 1 |
| `Shift-Alt-2` | Move to workspace 2 |
| `Shift-Alt-3` | Move to workspace 3 |
| `Shift-Alt-4` | Move to workspace 4 |

### 6.4 Display Management

Move the focused window between physical displays.

```
# Move window between displays
shift + alt - s : \
  yabai -m window --display west; \
  yabai -m display --focus west
shift + alt - g : \
```

```
  yabai -m window --display east; \
  yabai -m display --focus east
```

## 6.5 Window Resizing

Resize windows in 50-pixel increments using `Ctrl-Alt` and directional keys.

```
# Resize windows
lctrl + alt - h : \
  yabai -m window --resize left:-50:0; \
  yabai -m window --resize right:-50:0
lctrl + alt - j : \
  yabai -m window --resize bottom:0:50; \
  yabai -m window --resize top:0:50
lctrl + alt - k : \
  yabai -m window --resize top:0:-50; \
  yabai -m window --resize bottom:0:-50
lctrl + alt - l : \
  yabai -m window --resize right:50:0; \
  yabai -m window --resize left:50:0

# Equalise all window sizes
lctrl + alt - e : yabai -m space --balance
```

| Shortcut | Action |
|---|---|
| `Ctrl-Alt-H` | Shrink horizontally |
| `Ctrl-Alt-J` | Grow vertically |
| `Ctrl-Alt-K` | Shrink vertically |
| `Ctrl-Alt-L` | Grow horizontally |
| `Ctrl-Alt-E` | Equalise all windows |

## 6.6 Layout Rotation and Mirroring

Rotate the entire layout or mirror it across an axis.

```
# Rotate layout
alt - r : yabai -m space --rotate 270
shift + alt - r : \
  yabai -m space --rotate 90

# Mirror on X and Y axis
shift + alt - x : \
  yabai -m space --mirror x-axis
shift + alt - y : \
```

```
  yabai -m space --mirror y-axis
```

## 6.7 Gaps, Floating, and Fullscreen

Toggle gaps, float individual windows, and enter fullscreen mode.

```
# Toggle gaps
lctrl + alt - g : \
  yabai -m space --toggle padding; \
  yabai -m space --toggle gap

# Float or unfloat a window
shift + alt - space : \
  yabai -m window --toggle float; \
  yabai -m window --toggle border

# Fullscreen
alt - f : \
  yabai -m window --toggle zoom-fullscreen
shift + alt - f : \
  yabai -m window --toggle native-fullscreen
```

## 6.8 Application Launcher and Service Control

Open a terminal and restart yabai from the keyboard.

```
# Open iTerm2
alt - return : "${HOME}"/bin/open_iterm.sh

# Restart yabai
shift + lctrl + alt - r : \
  /usr/bin/env osascript <<< \
    "display notification \
    \"Restarting Yabai\" \
    with title \"Yabai\""; \
  launchctl kickstart -k \
    "gui/${UID}/homebrew.mxcl.yabai"
```

## 6.9 Things to Watch Out For

1. **SIP must be partially disabled.** Yabai's scripting addition requires System Integrity Protection to be partially disabled. Without it, yabai can manage window focus but cannot move or resize windows belonging to other applications.

2. **macOS updates may re-enable SIP.** After a major macOS update, check whether SIP has been re-enabled. If yabai stops managing windows after an update, this is the most likely cause.

3. **Some applications resist tiling.** System Preferences, Finder dialogs, and certain Electron apps may not respond to yabai commands. Use `yabai -m rule --add app="System Preferences" manage=off` to exclude specific applications.

4. **`fn` key behaviour varies by keyboard.** The `mouse_modifier fn` setting assumes a standard Apple keyboard. On external keyboards, `fn` may not be recognised; consider using `ctrl` or `cmd` instead.

5. **skhd conflicts with system shortcuts.** Some `Alt` combinations are already bound by macOS or specific applications. Test each shortcut to confirm it does not conflict with existing bindings.

Figure 3: A wide-angle view of a workspace with multiple monitors arranged side by side.

*Reflecting on a workspace that now manages itself.*

## 6.10 Lessons Learnt

### 6.10.1 Conceptual Understanding

- A tiling window manager fundamentally changes the relationship between the user and the desktop by eliminating manual window placement entirely.
- Binary space partitioning produces balanced layouts automatically, but the split direction alternates, which can produce unexpected layouts with many windows.
- The `focus_follows_mouse` setting reduces the number of deliberate focus-switching actions, which compounds into significant time savings over a workday.
- Window gaps serve a functional purpose beyond aesthetics: they make window boundaries visible, which helps when focus follows the mouse.

### 6.10.2 Technical Skills

- Yabai's configuration is a bash script, which means shell variables, loops, and conditionals can be used to create dynamic layouts.
- skhd uses a simple grammar for modifier keys (`alt`, `shift`, `lctrl`) that is more readable than the hex-based keycodes used by other hotkey daemons.
- The `yabai -m rule` command allows per-application exceptions, which is essential for handling dialog boxes and floating utilities.
- Display management commands (`--display west`, `--display east`) require multiple monitors to be connected; they fail silently on single-monitor setups.

### 6.10.3 Gotchas and Pitfalls

- Restarting yabai from skhd requires using `launchctl kickstart` rather than `yabai --restart-service` to ensure the scripting addition reloads correctly.
- The `window_border` setting can cause visual artefacts on some macOS versions; disable it if borders flicker or appear misaligned.
- Changing `layout` from `bsp` to `float` on a space does not restore window positions; windows remain where the tiling manager placed them.
- The `auto_balance on` setting redistributes space equally after every window event, which can be disorienting for practitioners who prefer a primary window with smaller side panes.

## 6.11 Limitations

- Yabai requires SIP to be partially disabled, which some organisations prohibit on managed macOS devices.
- The scripting addition must be reloaded after every macOS update, adding maintenance overhead.
- Yabai does not support Wayland or Linux; it is exclusively a macOS tool.
- Complex multi-monitor setups with different resolutions and scaling factors can produce inconsistent gap sizes.
- There is no graphical configuration tool; all settings must be edited in text files and reloaded manually.

- Applications that create their own window management (e.g., Emacs frames, some Java applications) may conflict with yabai's layout decisions.

### 6.12 Opportunities for Improvement

1. Create per-workspace layout rules so that workspace 1 uses BSP while workspace 2 uses a stacked layout for reference documents.
2. Add application-specific rules using `yabai -m rule` to exclude floating utilities, dialog boxes, and system preference panes.
3. Write a shell script that detects the number of connected displays and adjusts gap sizes accordingly.
4. Integrate with spacebar or SketchyBar for a status bar that displays workspace information.
5. Create workspace-specific shortcuts that launch predefined application layouts (e.g., "research mode" with Vim, a terminal, and a PDF viewer).
6. Explore `yabai -m signal` to trigger scripts when specific window events occur, such as automatically floating new dialog boxes.

## 7 Wrapping Up

Configuring yabai and skhd transformed my macOS desktop from a manually managed collection of overlapping windows into a keyboard-driven tiling environment. The initial investment of time (perhaps two hours of configuration and testing) has been repaid many times over in reduced context switching and fewer mouse interruptions.

The most important lesson from this process is that window management is a solved problem. The Linux community figured this out years ago with i3wm, bspwm, and similar tools. Yabai brings the same philosophy to macOS with minimal compromise.

For anyone starting with yabai, begin with just the navigation shortcuts (`Alt-H/J/K/L`) and the BSP layout. Add workspace management and resizing shortcuts only after the basic navigation feels natural. Trying to learn thirty keybindings at once is a recipe for frustration.

In conclusion, four points merit emphasis. First, BSP tiling with 12-pixel gaps provides a clean, automatic layout for most development workflows. Second, Vim-style directional navigation (`H/J/K/L`) transfers muscle memory from text editing to window management. Third, `focus_follows_mouse` eliminates a significant source of unnecessary keystrokes throughout the day. Fourth, version-controlling `.yabairc` and `.skhdrc` makes the entire desktop configuration portable and reproducible.

## 8 See Also

**Related posts:**

- "Configure the Command Line for Data Science Development" (terminal and shell configuration)
- "Setting Up R, Vimtex, and UltiSnips in Vim" (complementary Vim configuration)

**Key resources:**

- Yabai GitHub repository
- skhd GitHub repository
- Tiling window manager (Wikipedia)
- Tiling macOS: Moving from i3wm to Yabai – Steven Braun
- macOS Yabai Configuration – Bryce Smith
- Configuring Yabai for Focus – Evan Travers

## 9 Reproducibility

This configuration was developed and tested on macOS Ventura (13.x) and macOS Sonoma (14.x) with yabai 6.x and skhd 0.3.x. The following files constitute the complete configuration:

| File | Purpose |
|---|---|
| `~/.yabairc` | Tiling layout and behaviour |
| `~/.skhdrc` | Keyboard shortcut mappings |

To reproduce the environment:

```
# Install yabai and skhd
brew install koekeishiya/formulae/yabai
brew install koekeishiya/formulae/skhd

# Copy configuration files
cp yabairc ~/.yabairc
cp skhdrc ~/.skhdrc
chmod +x ~/.yabairc

# Start services
yabai --start-service
skhd --start-service
```

Note that SIP must be partially disabled before yabai's scripting addition will function. Consult the yabai wiki for platform-specific instructions.

## 10 Let's Connect

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas
- **Email:** rgtlab.org/contact

I would enjoy hearing from you if:

- You spot an error or a better approach to any of the code in this post.
- You have suggestions for topics you would like to see covered.
- You want to discuss R programming, data science, or reproducible research.
- You have questions about anything in this tutorial.
- You just want to say hello and connect.

## 10.1 Related posts in this cluster

This post is part of the *Workflow Construct* series. Recommended reading order:

1. Post 15: A Workflow Construct for the Modern Data Scientist
2. Post 16: Unix Command-Line Workspace Setup for Data Science
3. Post 17: Multi-Laptop macOS Bootstrap
4. Post 18: Setting Up Git for Data Science Workflows
5. Post 19: Setting Up Neovim as a Data Science IDE
6. Post 20: Extending the R-Vim Workflow with LaTeX
7. Post 21: Modern CLI Replacements for the Shell Layer
8. Post 22: LLM-Augmented Editing for the Workflow Construct
9. **Post 23: Configuring Yabai as a Tiling Window Manager** (this post)
10. Post 24: A pocket terminal with ttyd and Tailscale
11. Post 25: Install Linux Mint on a MacBook Air